Cost Optimization Status

Job Names	Environment	Assignee	Status	Short Desc
ehcuseim-adf-admbuild-pd1	ADB	@Priyanka Yadav	In progress	Pipeline: **pl_adm_build_submaster (pl_extractdata_xref_dailyrefresh)**, Notebook: **convert_xref_tables_delta_dailyrefresh** [convert_xref_tables_delta_dailyrefresh - Databricks] **Problem 1:** Deleting directories before overwriting Delta tables (using dbutils.fs.rm(...)) adds unnecessary overhead. Recursive deletes in ADLS incur high transaction costs and increase execution time. **Solution:** Avoid manual directory deletion. Use **mode("override")** in your write operation, which already replaces table data **Problem 2:** `drop table if exists` + `create table` causes extra metastore ops and can break permissions/lineage. **Solution:** Use `CREATE OR REPLACE TABLE` (managed) or `REPLACE`

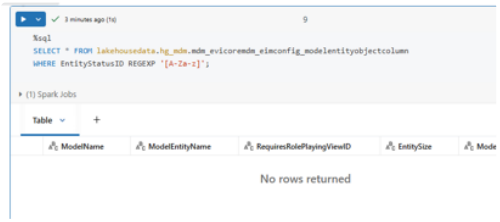
```
if (entityname == "xref_diagnosis_history" || entityname == "xref_diagnosiscodeset_history" || entityname ==
"xref_programdim" || entityname == "xref_cpttoproduct_changeset" || entityname == "xref_queuecodetocodegroupxref" ||
entityname == "xref_imageonesocsiteexclusions" || entityname == "xref_imageonesocinsurer" || entityname ==
"xref_imageonesocstateexclusions" || entityname == "xref_workflowsteps")
  {
    if (entityeimmdmpathexists == true)
    {
      //delete the files if exists
      dbutils.fs.rm("/mnt/eimmdm/hg_xref_extract/"+entityname+"/", true);
    }
    //drop table if exists
      spark.sql("drop table if exists eimmdm."+entityname)

    // save the dataframe as delta table table in the target location
      entitydmzextract_parquet_df.write.format("delta").mode("overwrite").option("overwriteSchema","true").save("/mnt/
      eimmdm/hg_xref_extract/"+entityname+"/")

      spark.sql("create table eimmdm."+entityname+" using delta location '/mnt/eimmdm/hg_xref_extract/"+entityname+"/'")
  }
```

Pipeline: **pl_adm_build_submaster**, Notebook: **unb_adm_encounter_build_create_eimconfig_modelentityobjectcolumn_table**
[unb_adm_encounter_build_create_eimconfig_modelentityobjectcolumn_table - Databricks]

- **Problem 1:** Applying lower() on every column is costly because:
  - It prevents predicate pushdown.
  - It forces full column transformations.
- **Optimization:**
  - If Column is ID, remove lower()



Pipeline:pl_extractdata_xref_dailyrefresh notebook:
[build_xref_delta_tables_dailyrefresh - Databricks]

- For Unity Catalog-managed Delta tables, use `INSERT OVERWRITE` instead of `TRUNCATE + INSERT` for full refresh operations. This approach is atomic, avoids intermediate empty states, reduces

metadata overhead, and scales efficiently as table size grows.

Pipeline:pl_admbuild_master_sourcesytem

Notebooks:
1.unb_adm_encounter_build_imageone_factencounterprocedureline_fepviewenablement - Databricks

2. unb_adm_encounter_build_isaac_factencounterprocedureline_fepviewenablement - Databricks

Problem 1:

`autoCompact = false` = **no automatic small-file merging** →you must manually optimize.

Solution:
`autoCompact = true` = Delta will **auto-merge small files** after writes ->better performance for frequent small updates.

Pipeline:act_ep_pl_admbuild_isaac_prebuild and act_ep_pl_admbuild_imageone_prebuild

Notebook: **All 8 notebooks from both pipelines**

Problem 1: Calling `UPPER(TRIM(LTRIM/RTRIM))` and `COALESCE/NULLIF` many times. Each call adds CPU cost.

Solution: Normalize once per column, then reuse.

-- In a CTE:

				normalized_carriername AS UPPER(TRIM(COALESCE(NULLIF(c.carrier, ''), 'unknown')) Prbblem 2: **getArgument is deprecated:** The getArgument function in SQL is deprecated and will be removed from DBR 17.0 onwards. Solution: Use the new parameter marker syntax (:param) instead. Pipeline: act_ep_pl_admbuild_master_intergerkeys Notebook:unb_adm_encounter_build_generate_integerkeys_master (for issac, imageone and paidlives) Problem 1: .take(list_of_objects_to_create_intkeys_df .count.toInt **Two separate Spark actions** • **count** triggers a full job: scans all partitions, aggregates the count. • **take(n)** triggers another job that reads until *n* rows are collected, this can cause partition reads and sometimes multi-stage operations depending on ordering and data distribution. • **Double scheduling, double IO, double overhead** Solution: 1. collect: **one job** doing the same scan once →1–3 seconds. 2. map: **linear time** → sub-second. **30–60% wall-time reduction** in the orchestration pre-step, and lower cost

				due to the removed duplicate Spark action. val objs = list_of_objects_to_create_intkeys_df .as[cls_list_of_objects_to_create_intkeys] .collect()
ehcuseim-adf-adm-cmsbuildpublish-pd01	ADB	@Rohid Shaik	In progress	1. **Single Data Read**: Read the source data only once and use **TRUNCATE** instead of **DROP**, followed by appending data through on-prem stored procedures. (act_nb_cms_publish_isaac_reprocess_to_sqldw) 2. **Optimize Stored Procedures**: Replace repetitive **DROP and CREATE TABLE** operations with **TRUNCATE and INSERT** for better efficiency. (act_if_check_dmz_extract_isaac, act_sp_factcmsencounter_isaac_build) 3. **Pre-Cleaning via Views**: Remove `LOWER(TRIM())` from merge operations by creating **views** that handle trimming beforehand, improving merge performance. (act_nb_cms_isaacbuild) 4. **Caching for Performance**: Cache the `fact_encounter_clean` table to significantly reduce read time and improve overall processing speed. (act_nb_cms_isaacbuild) 5. **Z-Order Optimization**: Apply **OPTIMIZE ZORDER BY encounter_id** on the `factcmsencounter`

				table to enhance merge efficiency. (act_nb_cms_isaacbuild) 6. **Use Views Where Possible**: If tables are not required in subsequent notebooks, create **views instead of tables** to reduce storage and maintenance overhead (pending implementation). 7. Removing unnecessary display action of dataframes(act_nb_ingest_data_to_delta_table)
eheu2eim-adf-nlp-pd01	ADB	@Harsha Tammisetti	In progress	Cancelled
eheu2eim-adf-inbound-fileingestion-pd1	ADB	@Vikash Kumar	In progress	
eheu2eim-adf-membermodel-flatload-pd01	ADB	@Akash Janjal	In progress	Pipeline: pl_membermodel_extract_Master This pipeline runs a notebook **unb_mbr_master** multiple time through foreach activity. Each time it is trying to query **lakehouse.bronzework.vw_lza_dx_eligibilityprocessing** by applying CarrierName and CarrierMarket filter. This table is huge and querying it again and again would consume lot of time and cost. **Solution**: Create a common notebook outside foreach which will generate an intermediate DMZ table from lakehouse.bronzework.vw_lza_dx_eligibilityprocessing by specifying necessary

				CarrierName and carrierMarket in where clause. Then use this DMZ table in notebook unb_mbr_master instead of lakehouse.bronzework.vw_lza_dx_eligibilityprocessing. Expecting 60% cost saving
ehcuseim-adf-platinumbuild-pd01	ADB	@Swapnil Kanade	In progress	
eheu2eim-adf-adm-mstrfactcustencreport-pd01	ADB	@Priyanka Yadav	In progress	This pipeline is having 2 waits activities which depends on CMS and TAT publish, other than wait activities, other activities are taking seconds to complete.
ehcuseim-adf-goldbuild-pd1	ADB	@Harsha Tammisetti	In progress	1. **NB -** unb_adm_encounter_build_factpacconsolidated_changeset: - 35 mins There are about 54 separate INSERT statements into dmz.unb_FPC_build_changeset_work. Each insert pulls data from different source tables with specific conditions. Instead of multiple inserts, we can: Create individual DataFrames for each source table. Apply all required conditions and transformations on these DataFrames. Combine them into one consolidated DataFrame (or a few logical batches). Perform a single bulk insert or MERGE into the target table. 2. **NB** - unb_adm_encounter_ep_xref_ePPACReportingStatusCrosswalk - around 7 mins average time Currently, the table is hard-coded and

				recreated daily, which is not ideal because this table is intended to be created only once. 3. **NB** - unb_adm_encounter_ep_build_authstatus_history -- Most notebooks use CREATE OR REPLACE for the table, but it's better to use overwrite instead. Applying LOWER and TRIM on silver tables every time slows performance This table is used in many notebooks (like unb_FPC_build_changeset_work), so caching it will improve speed. We don't know the data size, but partitioning was not used anywhere 4. NB - unb_adm_encounter_ep_build_factpac consolidated -- `unb_dmz_pac_createtemptables` - Around 40 tables created for every run better the use overwrites
ehcuseim-adf-admin-maintenance-pd01	ADB	@Rohid Shaik	In progress	Analyzing the pipeline